Symbolic Control Project

Computation Theory Course

UM6P College of Computing

Professor: Adnane Saoud **Program:** Computer Engineering
Session: Fall 2026

Team Information

Member 1	Omar Tsouli
Member 2	Aymane Raiss
Member 3	Amine Chrif
Member 4	Rayane Khaldi
Member 5	Younes Lougnidi
Repository Link	`https://github.com/Younes-Lougnidi/Symbolic-Control-Project`

Contents

I Mobile Robot Symbolic Control **3**

1 Introduction **3**

1.1 Context and Objective . . . 3

1.2 Problem Overview . . . 3

1.3 Work Plan . . . 4

1.4 Report Organization . . . 4

2 Symbolic Abstraction and Reachability Analysis **4**

2.1 Theory . . . 4

2.1.1 From a continuous state space to a finite model . . . 5

2.1.2 Partitioning states and selecting commands . . . 6

2.1.3 Defining transitions through over-approximation . . . 6

2.1.4 Two ways to enclose a reachable set . . . 7

2.1.5 Applying the method to the mobile robot: Section 3.1 . . . 9

2.1.6 Robot discretization: the beginning of Section 3.2 . . . 12

2.1.7 Derivative bounds and the size of the enclosure . . . 12

2.2 Implementation . . . 13

2.2.1 Overview of the implementation . . . 13

2.2.2 Subtask 1: `coord2state.hpp` . . . 13

2.2.3 Subtask 2: `state2coord.hpp` . . . 14

2.2.4 Subtask 3: `dynamics.hpp` . . . 15

2.2.5 Assembly: `symb_abs.cpp` . . . 17

3 Symbolic Controller Synthesis **20**

3.1 Reachability . . . 20

3.1.1 Theory . . . 20

3.1.2 Implementation . . . 20

3.2 Safety . . . 20

3.2.1 Theory . . . 20

3.2.2 Implementation . . . 20

3.3 Automaton-Based Specifications . . . 20

3.3.1 Region Labeling and Mission Rules . . . 20

3.3.2 Controller Synthesis and Command Lookup 20
3.3.3 Controller Concretization 20
3.3.4 Simulation and Trajectory Plots 20
3.3.5 Integration and Testing 20

4 Concretization and Simulation **20**

5 Conclusion **20**

Part I

Mobile Robot Symbolic Control

1 Introduction

1.1 Context and Objective

In this first part of the project, we apply symbolic control to a **mobile robot navigation problem**: a unicycle-type robot, subject to bounded disturbances and to state and input constraints, must satisfy a temporal specification over several regions of its workspace. Our work follows the approach presented in the reference article by Girard, Meyer and Saoud, *Approches symboliques pour le contrôle des systèmes non linéaires* (Techniques de l'Ingénieur, 2024), and the project statement provided in class.

1.2 Problem Overview

The robot evolves in the plane. Its state is its position and heading $x(t) = (x_1(t), x_2(t), x_3(t))$, and it is controlled through its linear and angular velocities $u(t) = (u_1(t), u_2(t))$. In discrete time, and under a bounded disturbance $w(t)$, its dynamics take the generic form

$$x(t+1) = f(x(t), u(t), w(t)), \qquad x(t) \in X,\ u(t) \in U,\ w(t) \in W, \tag{1}$$

where X, U and W are compact sets encoding the state constraints, the input constraints and the disturbance bounds, respectively. The model and discretization choices are detailed in Section 2.

We consider three control specifications over the workspace, of increasing complexity. The regions $R_1, \ldots, R_4$ are disjoint subsets of the state space.

1. **Reachability (going from one state to another).** Starting from an initial region, the robot must reach a given target region in finite time.
2. **Safety (avoiding an obstacle).** The robot must never enter the obstacle region, at any time.
3. **Temporal specification.** This specification combines reachability and safety over the four regions:

   > *"Go to R_1 or to R_2, but only to one of them, then to R_3, while avoiding R_4 all the way."*

The first two are elementary specifications, for which a static state-feedback controller is sufficient. The third goes beyond classical stability objectives: it is a co-safe LTL property that can be captured by a finite-state automaton, and its controller requires memory.

1.3 Work Plan

To structure the work, we divided the project into four consecutive steps. Each step builds on the previous one, which lets us validate the theory before implementing it and validate the implementation before using it to synthesize a controller.

1. **Abstraction and reachability: theory.** We identified the parts of the reference article that formalize symbolic abstraction and reachability analysis, namely Section 1.1 (abstraction methodology) and Sections 3.1 and 3.2 (model, specification, abstraction and controller synthesis for the mobile robot). The goal of this step is to understand and document how the state and input spaces are discretized and how the symbolic transition function is obtained by over-approximating reachable sets.
2. **Abstraction and reachability: implementation.** We will implement the methods described in the first step, namely the partition of the state space, the quantization of the inputs, and the computation of the transition function through reachability analysis. This implementation is the foundation on which all subsequent steps are built.
3. **Controller synthesis: theory and implementation.** We will study the synthesis methods presented in Section 2 of the article, in particular the treatment of automaton-based specifications through a product between the symbolic model and the specification automaton. We will then implement the synthesis algorithm on top of the abstraction developed in step 2.
4. **Concretization and real-time control.** Finally, we will bring the work together by going from the symbolic controller to a controller applicable to the real robot dynamics (Section 1.2 of the article), and validate it through numerical simulations (Section 3.3).

1.4 Report Organization

This part of the report follows these four steps. Section 2 presents the model, the specification, and the abstraction and reachability analysis (steps 1 and 2), Section 3 covers controller synthesis (step 3), and Section 4 covers the concretization of the controller and the simulation results (step 4). At this stage, only the theory of abstraction has been documented; implementation and subsequent stages remain to be completed.

2 Symbolic Abstraction and Reachability Analysis

2.1 Theory

First meeting — 7 October 2026

During our first meeting on 7 October 2026, we discussed the abstraction method step by step. We first worked through Section 1.1 of the reference article, then connected its definitions to the mobile robot in Sections 3.1 and 3.2. Our objective was to understand what each symbolic state represents, how uncertainty produces several possible successors, and how reachable sets can be enclosed safely. This section records our understanding at

the end of that meeting.

2.1.1 From a continuous state space to a finite model

We started from the discrete-time system

$$x(t+1) = f(x(t), u(t), w(t)), \qquad x(t) \in X, \quad u(t) \in U, \quad w(t) \in W,$$

where X, U and W are compact constraint sets. Here "continuous" refers to the possible values of the state, whereas time is discrete. We understood symbolic abstraction as replacing this infinite collection of states and commands by a finite description:

$$\xi(t+1) \in g(\xi(t), \sigma(t)), \qquad \xi(t) \in X_s, \quad \sigma(t) \in U_s.$$

The transition map g is set-valued. Even after choosing a command, we do not know the exact point within the current cell or the disturbance. Several successors must therefore remain possible.

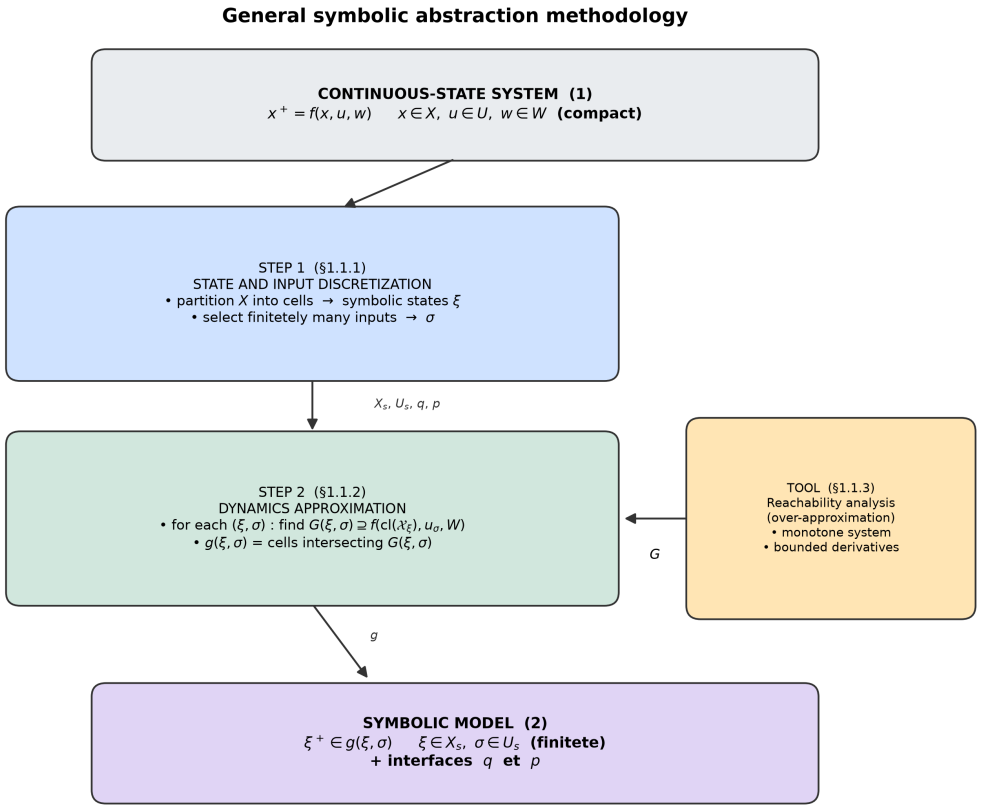


Figure 1: The two stages we studied: discretizing states and inputs, then enclosing their possible next states.

2.1.2 Partitioning states and selecting commands

We partition X into m cells $\mathcal{X}_1, \ldots, \mathcal{X}_m$ and associate one symbolic state ξ_i with each cell. An additional state ξ_0 represents $\mathcal{X}_0 = \mathbb{R}^n \setminus X$:

$$X_s = \{\xi_0, \xi_1, \ldots, \xi_m\}.$$

Cell boundaries need a consistent assignment so that each point in X belongs to exactly one cell. The abstraction interface is

$$q(x(t)) = \xi_i \quad \text{when } x(t) \in \mathcal{X}_i.$$

For the inputs, we select finitely many admissible values $u_1, \ldots, u_l \in U$. We then define

$$U_s = \{\sigma_1, \ldots, \sigma_l\}, \qquad p(\sigma_j) = u_j.$$

The distinction that helped us most is that a symbolic state represents *an entire set of points*, while a symbolic command represents *one selected input value*. The map q reads the robot state at the symbolic level; p translates a symbolic command into a real input.

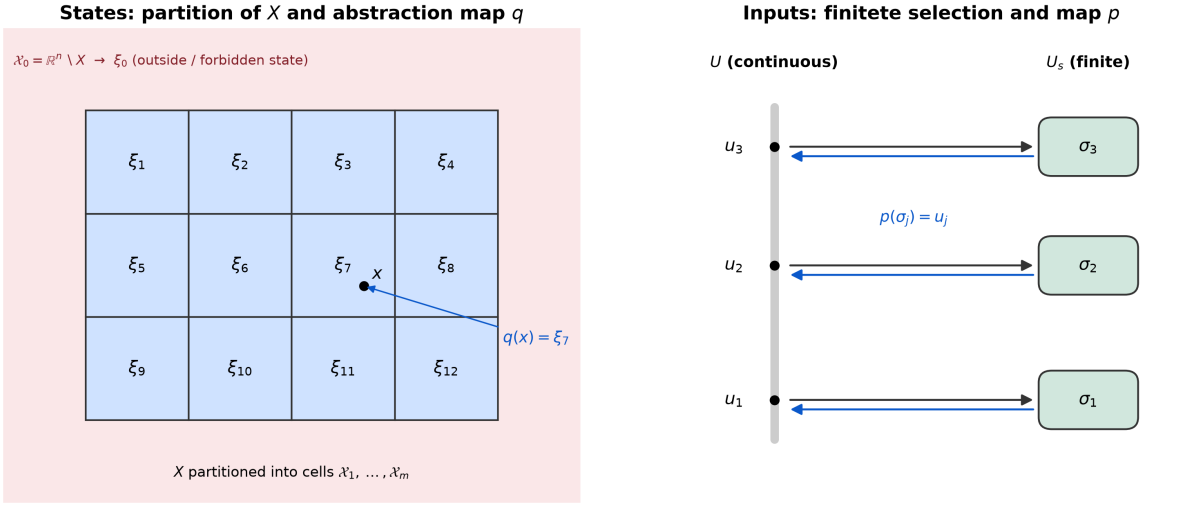


Figure 2: State cells, the outside state, and the interfaces q and p. The small grid illustrates the definitions; it is not the robot's full grid.

2.1.3 Defining transitions through over-approximation

For a fixed cell and command, the one-step reachable set is

$$\mathcal{R}(\xi, \sigma) = \{f(x(t), u_\sigma, w(t)) : x(t) \in \mathrm{cl}(\mathcal{X}_\xi),\ w(t) \in W\}.$$

The closure includes the cell's boundary when computing a safe enclosure. Computing this set exactly is often difficult for nonlinear dynamics, so we construct a simpler set $G(\xi, \sigma)$ satisfying

$$\mathcal{R}(\xi, \sigma) \subseteq G(\xi, \sigma). \tag{2}$$

We retain every cell touched by that enclosure:

$$g(\xi, \sigma) = \{\xi' \in X_s : \mathcal{X}_{\xi'} \cap G(\xi, \sigma) \neq \varnothing\}. \tag{3}$$

In particular, if G intersects the complement of X, the outside state is a possible successor. This makes a possible violation of the state constraint visible in the finite model.

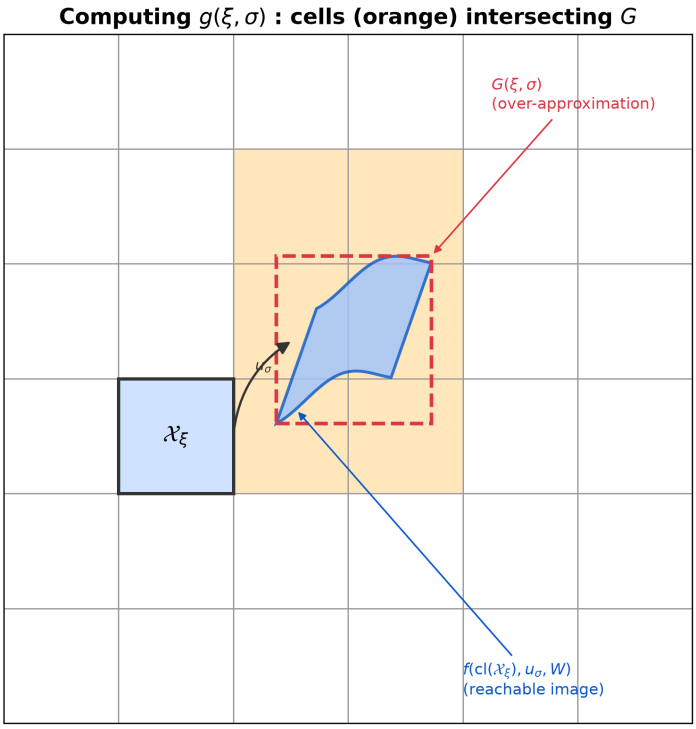


Figure 3: The source cell is blue, the reachable image is enclosed by the dashed box G, and the orange cells become symbolic successors. This is a schematic example, not a robot simulation.

What we understood about uncertainty

We choose the command, but we cannot choose which successor occurs. The abstraction must cover every admissible disturbance and every point in the source cell. A larger enclosure is safe but may introduce transitions that the physical system cannot actually realize.

2.1.4 Two ways to enclose a reachable set

We compared two interval-based methods. All inequalities below are componentwise, and $[a, b]$ denotes a multidimensional box.

Monotone dynamics.

Following Section 1.1.3 of the article, we first considered a differentiable system for which every entry of the state and disturbance Jacobians is nonnegative:

$$\frac{\partial f_i}{\partial x_j(t)} \geq 0 \qquad \text{and} \qquad \frac{\partial f_i}{\partial w_k(t)} \geq 0 \qquad \text{(for all } i, j, k\text{)}.$$

This implies the following order-preserving property:

$$x(t) \le \widetilde{x}(t), \qquad w(t) \le \widetilde{w}(t)$$
$$\implies \quad f(x(t), u_\sigma, w(t)) \le f(\widetilde{x}(t), u_\sigma, \widetilde{w}(t)).$$

Here vector inequalities compare corresponding coordinates, rather than vector lengths. For example, $x(t) \le \widetilde{x}(t)$ means $x_j(t) \le \widetilde{x}_j(t)$ for every j.

We then take a cell and disturbance set represented by boxes:

$$\mathrm{cl}(\mathcal{X}_\xi) = [\underline{x}, \overline{x}], \qquad W = [\underline{w}, \overline{w}].$$

The vectors $\underline{x}$ and $\overline{x}$ contain the lower and upper coordinate bounds of the current cell; they are fixed bounds. Thus every admissible pair satisfies

$$\underline{x} \le x(t) \le \overline{x}, \qquad \underline{w} \le w(t) \le \overline{w}.$$

Applying the order-preserving property twice gives

$$\begin{aligned} f(\underline{x}, u_\sigma, \underline{w}) &\le x(t+1) = f(x(t), u_\sigma, w(t)) \\ &\le f(\overline{x}, u_\sigma, \overline{w}). \end{aligned}$$

Therefore we only need to compute two successors: one from the lower state and disturbance corners, and one from the upper corners. They define the enclosure

$$G(\xi, \sigma) = \big[f(\underline{x}, u_\sigma, \underline{w}), f(\overline{x}, u_\sigma, \overline{w})\big].$$

We understood why this is the *smallest enclosing box*: every reachable state lies between these two vectors, and both corner images are themselves reachable from the closed cell and disturbance box.

Bounded derivatives.

If monotonicity is unavailable, we instead bound the absolute values of the derivatives throughout the relevant domain:

$$\left|\frac{\partial f}{\partial x(t)}\right| \le D_x, \qquad \left|\frac{\partial f}{\partial w(t)}\right| \le D_w.$$

For the current cell $\mathrm{cl}(\mathcal{X}_\xi) = [\underline{x}, \overline{x}]$ and disturbance box $W = [\underline{w}, \overline{w}]$, we compute the centres and half-widths componentwise, as in the article:

$$x^*(t) = \frac{\underline{x} + \overline{x}}{2}, \quad \delta x = \frac{\overline{x} - \underline{x}}{2},$$
$$w^* = \frac{\underline{w} + \overline{w}}{2}, \quad \delta w = \frac{\overline{w} - \underline{w}}{2}.$$

Here $x^*(t)$ is the centre of the cell containing $x(t)$, not the exact measured state. For a symmetric disturbance box, $w^* = 0$. The centre of the enclosure is obtained by evaluating the dynamics at these two midpoints, under the selected command:

$$\begin{aligned} c &= f(x^*(t), u_\sigma, w^*) \\ &= f\left(\frac{\underline{x} + \overline{x}}{2}, u_\sigma, \frac{\underline{w} + \overline{w}}{2}\right). \end{aligned}$$

The half-width and the resulting enclosure are

$$r = D_x \delta x + D_w \delta w,$$
$$G(\xi, \sigma) = [c - r, c + r]. \tag{4}$$

We interpreted c as the nominal next state (because it is the state predicted at time $t+1$ when the current state is taken at the cell centre and the disturbance at its centre, under the selected command u_σ). It is the image of the input centres, not necessarily the midpoint of the exact reachable set. The vector r covers the variation inside the cell and the disturbance box. This method requires one evaluation of f at these centres once valid derivative bounds are available.

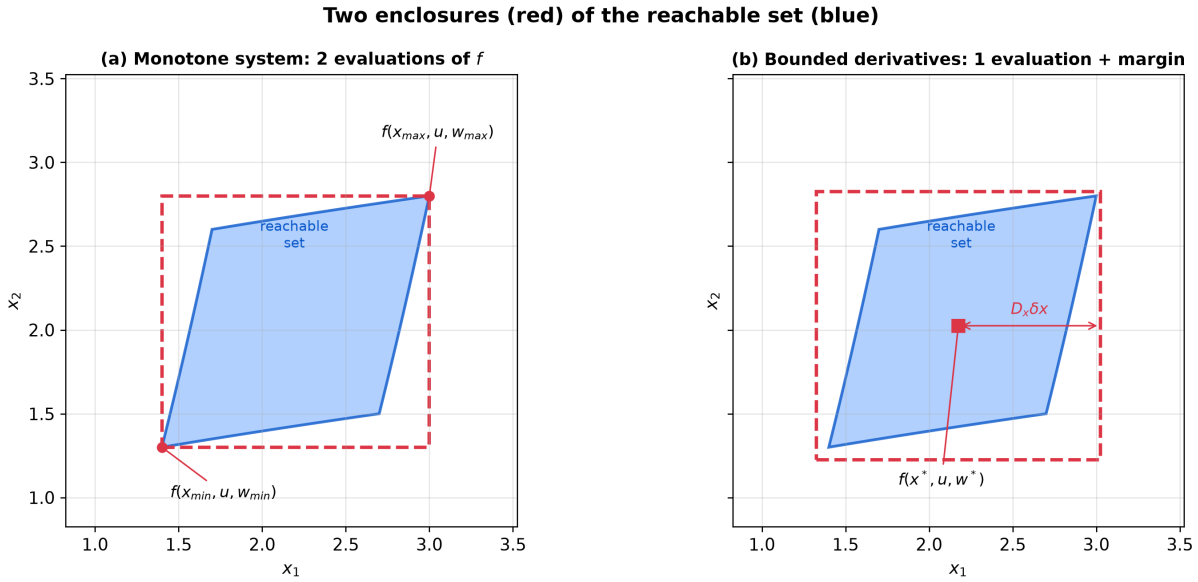


Figure 4: Comparison of reachability methods: corner bounds for a monotone system (left) and a centre-based enclosure using bounded derivatives (right).

We also discussed the balance between precision and size. Finer state partitions provide more detail but increase the number of symbolic states. Loose enclosures create more possible successors and can make later controller synthesis conservative. In particular, if every command allows a self-transition, the abstraction can fail to certify progress toward a target. This observation guided our understanding of the discretization choices.

2.1.5 Applying the method to the mobile robot: Section 3.1

We then connected these definitions to a unicycle robot. Its state is $x(t) = (x_1(t), x_2(t), x_3(t))$, where the first two coordinates describe position and $x_3(t)$ is the heading. The commands $u_1(t)$ and $u_2(t)$ are its linear and angular velocities. We consider the sampled model

$$\begin{aligned} x_1(t+1) &= x_1(t) + \tau(u_1(t)\cos x_3(t) + w_1(t)), \\ x_2(t+1) &= x_2(t) + \tau(u_1(t)\sin x_3(t) + w_2(t)), \\ x_3(t+1) &= x_3(t) + \tau(u_2(t) + w_3(t)). \end{aligned} \tag{5}$$

Here τ is the sampling period and $w(t)$ is a bounded disturbance. The position moves along the current heading, while the angular command changes that heading. The sine and cosine terms make this model nonlinear. The heading equation is written in a local angular coordinate; the course statement treats heading modulo 2π.

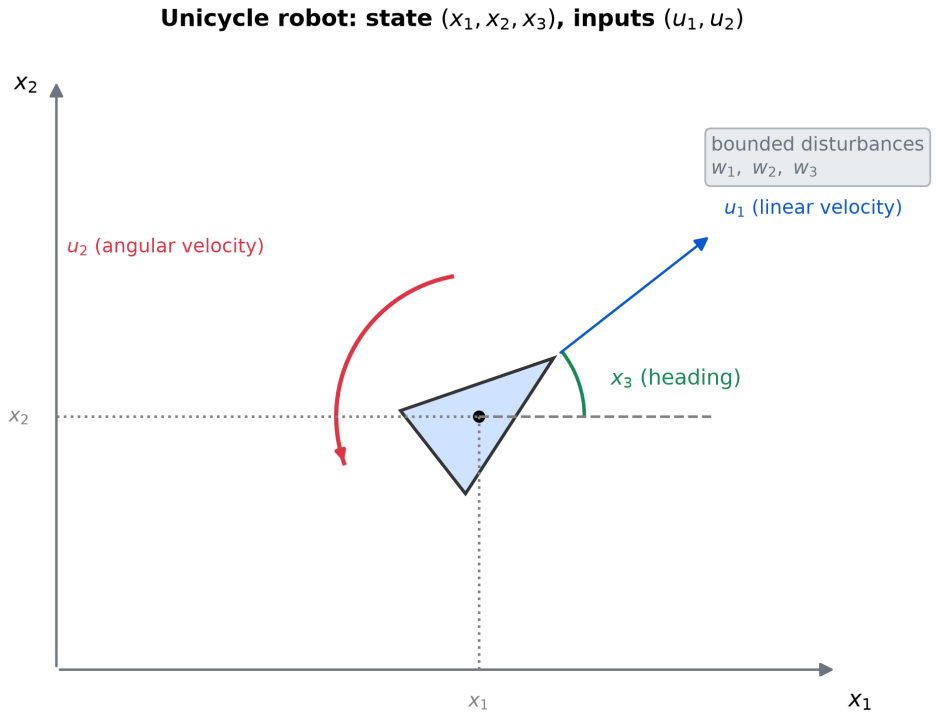


Figure 5: The robot state variables and control inputs.

The workspace is $X = [0, 10] \times [0, 10] \times [-\pi, \pi]$. We keep τ, the command bounds and disturbance half-widths symbolic in the derivation below.

Table 1: Regions defined in the reference article. The heading is unrestricted in every region.

Region	Position domain $(x_1(t), x_2(t))$	Role
R_1	$[4, 5] \times [8.5, 9.5]$	First destination, option 1
R_2	$[8.5, 9.5] \times [2, 3]$	First destination, option 2
R_3	$[2, 3] \times [0.5, 1.5]$	Subsequent destination
R_4	$[3, 7] \times [3, 7]$	Obstacle to avoid

We read the specification as choosing exactly one of R_1 and R_2, then reaching R_3, while avoiding R_4 throughout the task. This helped us see why the current position alone does not record progress: we also need to remember which first destination has been visited.

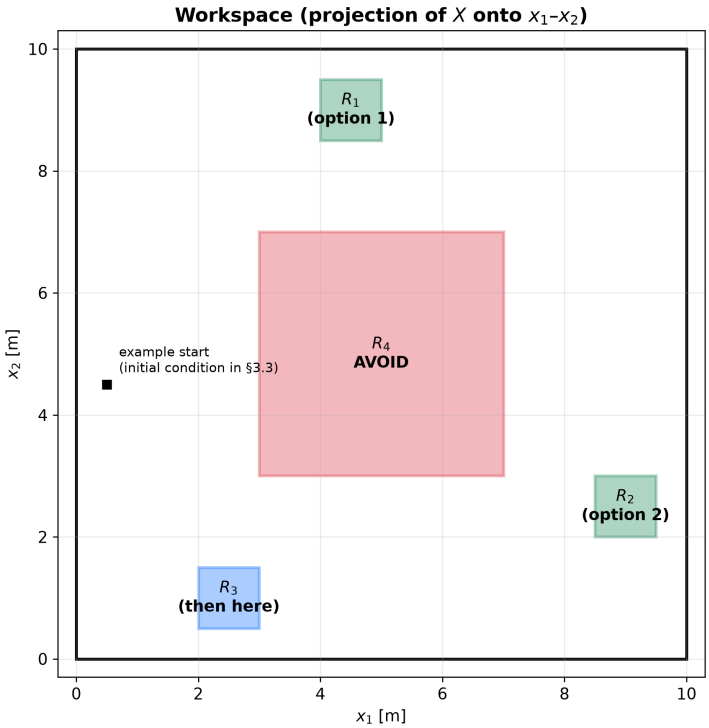


Figure 6: The four regions in the position plane. The marked starting point is illustrative; no closed-loop trajectory has been implemented by our group.

This memory is represented by a five-state automaton: state 0 means no first destination has been visited; states 1 and 2 record visiting R_1 or R_2; state 3 records completion; state 4 records failure. Entering the obstacle or visiting the other first destination before completion leads to failure. In this co-safe interpretation, completion and failure are absorbing states: the automaton evaluates the finite task up to completion, rather than imposing an additional perpetual safety requirement afterwards.

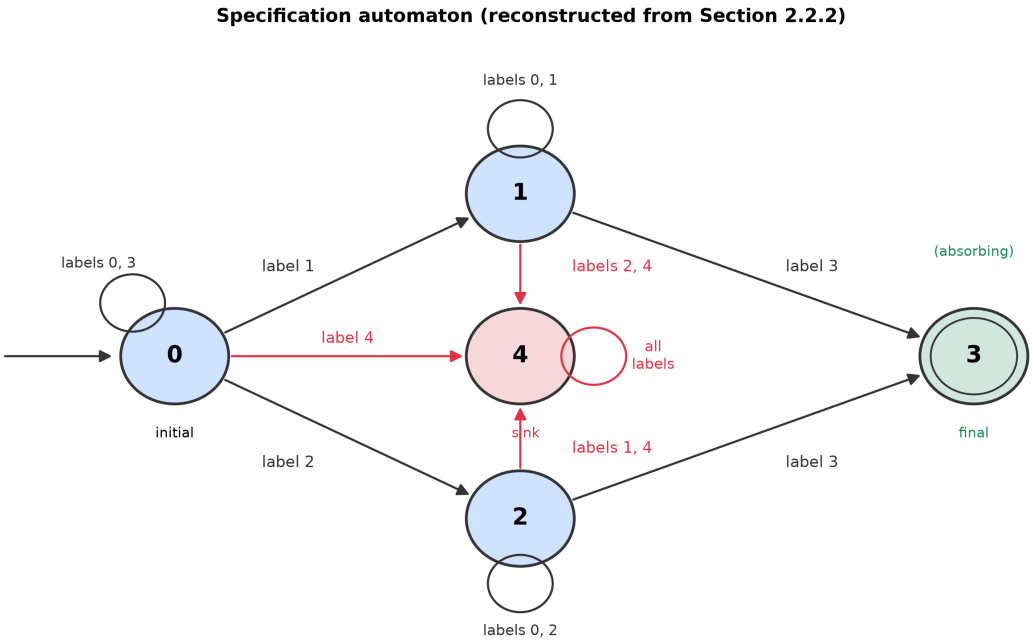


Figure 7: The specification automaton. Label i denotes region R_i; label 0 denotes a point outside all four regions.

2.1.6 Robot discretization: the beginning of Section 3.2

We followed the same abstraction steps with a uniform Cartesian partition:

$$N_1 = 100, \qquad N_2 = 100, \qquad N_3 = 30.$$

This gives $100 \times 100 \times 30 = 300\,000$ cells and $300\,001$ symbolic states when the outside state is included. Each cell has full widths

$$h = \left(0.1 \text{ m},\ 0.1 \text{ m},\ \frac{2\pi}{30} \text{ rad}\right), \qquad \delta x = \frac{h}{2}.$$

The selected input set combines three linear-velocity values and five angular-velocity values, giving $3 \times 5 = 15$ symbolic commands. These are the discretization sizes discussed in the article; they are not results of an implementation completed by our group.

2.1.7 Derivative bounds and the size of the enclosure

The robot is not globally monotone in the standard componentwise order:

$$\frac{\partial f_1}{\partial x_3(t)} = -\tau u_1(t) \sin x_3(t)$$

changes sign with the heading. We therefore understood why the bounded derivative method is used. Differentiating the robot model gives

$$\frac{\partial f}{\partial x(t)} = \begin{pmatrix} 1 & 0 & -\tau u_1(t) \sin x_3(t) \\ 0 & 1 & \tau u_1(t) \cos x_3(t) \\ 0 & 0 & 1 \end{pmatrix}, \qquad \frac{\partial f}{\partial w(t)} = \tau I_3.$$

With $|u_1|_{\max} = \max_{u \in U} |u_1|$, and using $|\sin x_3(t)| \leq 1$, $|\cos x_3(t)| \leq 1$, uniform bounds are

$$D_x = \begin{pmatrix} 1 & 0 & \tau|u_1|_{\max} \\ 0 & 1 & \tau|u_1|_{\max} \\ 0 & 0 & 1 \end{pmatrix}, \qquad D_w = \tau I_3.$$

Thus $G(\xi, \sigma) = [f(x^*(t), u_\sigma, w^*) - r, f(x^*(t), u_\sigma, w^*) + r]$. Because the grid is uniform and the same global derivative bounds are used for every cell and command, its half-widths are constant. Its centre changes with the cell and command; its size does not. This is the point at which we stopped during our first meeting:

Consequence: the same enclosure size for every cell

$$\boxed{\begin{aligned} r_1 &= \delta x_1 + \tau|u_1|_{\max}\,\delta x_3 + \tau\,\delta w_1, \\ r_2 &= \delta x_2 + \tau|u_1|_{\max}\,\delta x_3 + \tau\,\delta w_2, \\ r_3 &= \delta x_3 + \tau\,\delta w_3. \end{aligned}} \tag{6}$$

2.2 Implementation

Second meeting — 11 October 2026

During our second meeting we figured that not only did we need to master the theory, but also bring it to life via implementation. Down below is the overview of the work completed during that point.

2.2.1 Overview of the implementation

For the implementation of the symbolic abstraction, we need to divide our attention into three sub-problems. Two of them are conversion mechanisms: the conversion from the state coordinates to the state ID, and the inverse conversion from the state ID back to the actual state coordinates. Last but not least, we tackle the implementation of the dynamics, which, as an overview, covers the essential mathematical basis (Jacobians, bounds of each continuous input, etc.). Once these three subtasks are completed, we move on to the assembly. In other words, we use them to construct the whole logic behind the symbolic abstraction of our problem.

2.2.2 Subtask 1: `coord2state.hpp`

Let us start with the first subtask, namely the first `.hpp` file in our `symbolicAbstraction` folder, `coord2state`:

Listing 1: `coord2state.hpp`: from cell coordinates to a unique state ID.

```cpp
// C++ port of coord2state.m
// Grid coordinates (one index per dimension, 1-based) -> single state number (1-based).
// MATLAB: i = (I-1)' * p_x(1:length(I)) + 1
#pragma once
#include <array>

template <size_t N>
int coord2state(const std::array<int, N>& I, const std::array<int, N + 1>& p) {
    int i = 1;
    for (size_t k = 0; k < N; ++k) i += (I[k] - 1) * p[k];
    return i;
}
```

Explanation/Analysis.

This file only needs one function. It takes as a parameter an input array of coordinates, which are cell numbers, each coming from the continuous input parameters. For instance, if our robot is at (x_1, x_2, x_3), we ask: what does this translate to in our discrete representation? To do that, x_1 is mapped to a certain interval, as discussed in the discretization process, which gives us a certain cell, and the same goes for the other coordinates. So the goal here is to get those cell coordinates, together with a hash-like array which allows us to yield a unique ID for the exact cell our robot inhabits at the real coordinates (x_1, x_2, x_3).

The hash-like array is none other than (1, 100, 10 000, 300 000), but why? After the discretization, the x_1 values fit into 100 bins (intervals), the same holds for x_2, and there are 30 bins for x_3, so the mapping array is of the form (1, n_{x_1}, $n_{x_1}n_{x_2}$, $n_{x_1}n_{x_2}n_{x_3}$). We can read the numerical values as follows: if only x_1 changes, we move to the neighbouring cell (a step of 1). If we hold x_1 fixed and move along x_2, we move by 100 cells in the matrix. However, if we fix x_1 and x_2 and vary x_3, we move by blocks of 100×100 ($x_1 \times x_2$). Using this logic, we get the exact ID of the cell in the discrete grid of 300 000 cells built in the theory above.

2.2.3 Subtask 2: `state2coord.hpp`

Now moving on to the second concern. Given a state ID (a cell) among the 300 000 discrete states, how can we know from which bins it was yielded? The answer lies inside our second `.hpp` file, `state2coord`:

Listing 2: `state2coord.hpp`: from a state ID back to cell coordinates.

```cpp
// C++ port of state2coord.m
// Single state number (1-based) -> grid coordinates (one index per dimension, 1-based).
// MATLAB: I(k) = floor(mod(i-1, p(k+1)) / p(k)) + 1
#pragma once
#include <array>

template <size_t N>
std::array<int, N> state2coord(int i, const std::array<int, N + 1>& p) {
```

```
    std::array<int, N> I;
    for (size_t k = 0; k < N; ++k) I[k] = ((i - 1) % p[k + 1]) / p[k] +
        1;
    return I;
}
```

Explanation/Analysis.

This file also needs only one function. It takes as parameters the state ID and, of course, the hash-like array, in order to backtrack all the way to the origin, meaning the cell coordinates and not the real (x_1, x_2, x_3), because the mapping is not bijective in that sense. Using that formula, we return the wanted result.

Note

This function is used mainly when dealing with the concretization of the action input u, where we get a discrete input (from 1 to 15, since we have 3 discrete values for the linear velocity and 5 discrete values for the angular velocity). This is discussed below; let us save it for later.

2.2.4 Subtask 3: `dynamics.hpp`

The last, but not least, `.hpp` file is `dynamics`. In this file we set our dynamics function f, that of the real movement of the robot, the Jacobian matrices modelling the derivatives with respect to each of the function's parameters, as well as the several bounds and discretization settings:

Listing 3: `dynamics.hpp`: dynamics, Jacobian bounds, constraints and discretization settings.

```
// C++ port of dynamics.m
// Defines the system (unicycle), its bounds, and the abstraction grid
    sizes.
#pragma once
#include <array>
#include <cmath>

using Vec3 = std::array<double, 3>;
using Vec2 = std::array<double, 2>;
using Mat3 = std::array<std::array<double, 3>, 3>;
using Idx3 = std::array<int, 3>;
using Idx2 = std::array<int, 2>;

constexpr double PI = 3.14159265358979323846;

namespace dyn {

// Sampling time
constexpr double T = 1.0;

// System dynamics  x+ = f(x,u,w)
inline Vec3 f(const Vec3& x, const Vec2& u, const Vec3& w) {
    return {x[0] + T * u[0] * std::cos(x[2]) + T * w[0],
            x[1] + T * u[0] * std::sin(x[2]) + T * w[1],
```

```
            x[2] + T * u[1] + T * w[2]};
}

// Bounds on the Jacobian matrices (used to over-approximate reachable sets)
inline Mat3 Jf_x(const Vec2& u) {
    const double a = T * std::fabs(u[0]);
    return {{{1, 0, a},
             {0, 1, a},
             {0, 0, 1}}};
}

inline Mat3 Jf_w(const Vec2& /*u*/) {
    return {{{T, 0, 0},
             {0, T, 0},
             {0, 0, T}}};
}

// Constraints: first = lower bound, second = upper bound
constexpr Vec3 bound_x_lo = {0.0, 0.0, -PI};
constexpr Vec3 bound_x_hi = {10.0, 10.0, PI};
constexpr Vec2 bound_u_lo = {0.25, -1.0};
constexpr Vec2 bound_u_hi = {1.0, 1.0};
constexpr Vec3 bound_w_lo = {-0.05, -0.05, -0.05};
constexpr Vec3 bound_w_hi = {0.05, 0.05, 0.05};

// Abstraction parameters: number of cells per dimension
constexpr Idx3 n_x = {100, 100, 30};
constexpr Idx2 n_u = {3, 5};

}  // namespace dyn
```

Explanation/Analysis.

- **The robot function model.** As modelled mathematically, the function takes three parameters: a positional one, an actionable one, and a last one dedicated to the disturbances that might occur in each of the positional parameters due to sensor precision, etc. The function returns a vector containing the three equations modelling the robot's movements on the plane.
- **The Jacobian matrices.** This one is pretty self-explanatory: the derivative with respect to x depends on u and the constant T, whereas the derivative with respect to w only depends on the constant T. This is mathematically sound and clear.
- **The bounds.** Of course, they are given by the problem setting and are very essential for the dynamics of the robot (pretty self-explanatory too).
- **The discretization setting.** As stated in the theory part of this implementation, we opted for a $\{100, 100, 30\}$ discretization for x_1, x_2, x_3 respectively, yielding 300 000 cells/states. As for u, the discrete setting is $\{3, 5\}$, yielding 15 control actions.

The implementation of these subtasks truly simplifies the coding aspect of the core symbolic

abstraction. We will see how and why in our next segment, in which we discuss the process that leads us to it.

2.2.5 Assembly: `symb_abs.cpp`

Our next file is `symb_abs`, which is the first `.cpp` file.

Helper method.

First of all, we defined a helper method to bypass any matrix–vector multiplication.

Listing 4: Matrix–vector multiplication helper.

```
static Vec3 matvec(const Mat3& M, const Vec3& v) {
    Vec3 r{};
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j) r[i] += M[i][j] * v[j];
    return r;
}
```

Explanation/Analysis. This helper needs no further explanation.

State discretization.

Second is the state discretization. We discretize each of our positional parameters (x_1, x_2, x_3) like so:

Listing 5: State discretization.

```
Vec3 d_x;
for (int i = 0; i < 3; ++i) d_x[i] = (bound_x_hi[i] - bound_x_lo[i]) /
    n_x[i];
const std::array<int, 4> p_x = {1, n_x[0], n_x[0] * n_x[1], n_x[0] * n_x
    [1] * n_x[2]};
const int n_xi = p_x[3];
```

Explanation/Analysis.

- We have, for both x_1 and x_2, as we saw, a width of 0.1 for each state, and for x_3 it is $\frac{2\pi}{30}$, in conformity with the theory discussed above.
- We also build our hash-like array `p_x` in the same way as discussed before.
- We set the number of all possible states resulting from our discretization from `p_x[3]`, which, as discussed, is the correct thing to do.

Input discretization.

Third is the input discretization. We discretize the linear velocity as well as the angular velocity just like so:

Listing 6: Input discretization.

```
Vec2 d_u;
for (int i = 0; i < 2; ++i) d_u[i] = (bound_u_hi[i] - bound_u_lo[i]) / (
    n_u[i] - 1);
const std::array<int, 3> p_u = {1, n_u[0], n_u[0] * n_u[1]};
const int n_sigma = p_u[2];
```

Explanation/Analysis.

- Here it is good to note that we divide by `n_u[i] - 1`, to avoid the cardinality mistake coming from redundant values in two consecutive bins.
- Same as for the state, except that here we build `p_u`, which contains at its third slot `p_u[2]` the number of discrete inputs `n_sigma`.

Concretization function.

Fourth is the concretization function. As we discussed in the theory, for the actions (u_1, u_2) we need a concretization function, since our robot does not understand the discretized values $(1, \ldots, 15)$; it understands velocities. Here, we make use of our `state2coord` function and apply a transformation to know the actual yielder of each of the states.

Listing 7: Concretization function.

```
std::vector<Vec2> val_u(n_sigma);
for (int i = 1; i <= n_sigma; ++i) {
    Idx2 c = state2coord<2>(i, p_u);
    for (int k = 0; k < 2; ++k) val_u[i - 1][k] = bound_u_lo[k] + (c[k]
        - 1) * d_u[k];
}
```

Explanation/Analysis.

- As you can clearly see, we use all the possible discrete inputs, which are between 1 and 15 according to the problem specifications. Now the goal is to know what yielded each of them. So we build a 15×2 matrix, such that row i is the yielder of the state $i + 1$.
- The mathematical transformation is clear, and we confirmed that it works.

Disturbance setup.

Fifth is the disturbance setup to start the symbolic abstraction, and it goes like this:

Listing 8: Disturbance setup.

```
Vec3 d_w, w_c;
for (int i = 0; i < 3; ++i) {
    d_w[i] = bound_w_hi[i] - bound_w_lo[i];
    w_c[i] = 0.5 * (bound_w_lo[i] + bound_w_hi[i]);
}
```

Explanation/Analysis.

- We compute both `d_w` and the center `w_c`, since both are essential for the computation of the reachable set.

Simulation: computation of the reachable set.

Listing 9: Over-approximation of the reachable set.

```
Vec3 x_c_succ = f(x_c, u, w_c);
Vec3 a = matvec(Jf_x(u), d_x);
Vec3 b = matvec(Jf_w(u), d_w);
Vec3 reach_lo, reach_hi;
for (int i = 0; i < 3; ++i) {
    double dev = 0.5 * a[i] + 0.5 * b[i];
    reach_lo[i] = x_c_succ[i] - dev;
    reach_hi[i] = x_c_succ[i] + dev;
}
```

Explanation/Analysis. In the simulation we discuss a very important aspect of this symbolic abstraction. Without it, we would not be able to establish reachability, which is a pillar of our project. In a similar fashion to what we discussed in the theory, we need to multiply the Jacobians by their respective maximal step (the amplitude/length of the intervals). We apply this to the movement (x_1, x_2, x_3) and we also take the disturbance into account. This yields two bounds, a low and a high one, which determine the whole reachable set. Of course, this set might not represent *one* exact cell in our discretized grid. In this case, which is the most likely to happen each time, we take the intersection of our grid cells with that reachable set. This implies that our robot, in the discretized setting, can reach all of those cells.

Listing 10: Cells intersected by the reachable set.

```
Idx3 cmin, cmax;
for (int i = 0; i < 3; ++i) {
    cmin[i] = (int)std::floor((reach_lo[i] - bound_x_lo[i]) / d_x[i]) +
        1;
    cmax[i] = (int)std::ceil((reach_hi[i] - bound_x_lo[i]) / d_x[i]);
}
g[(size_t)(xi - 1) * n_sigma + (sigma - 1)] = {coord2state<3>(cmin, p_x)
    ,
                                               coord2state<3>(cmax, p_x)
                                                  };
```

Goal

With this process we build the transition function, which makes complete sense: from each cell, given an action, we compute the reachable states/cells. It is the same as

$$\sigma(q, a) \longrightarrow \{\, q_i \mid q_i \in \text{Reachable set} \,\}.$$

3 Symbolic Controller Synthesis

3.1 Reachability

3.1.1 Theory

3.1.2 Implementation

3.2 Safety

3.2.1 Theory

3.2.2 Implementation

3.3 Automaton-Based Specifications

3.3.1 Region Labeling and Mission Rules

3.3.2 Controller Synthesis and Command Lookup

3.3.3 Controller Concretization

3.3.4 Simulation and Trajectory Plots

3.3.5 Integration and Testing

4 Concretization and Simulation

5 Conclusion